



Computergraphik 2 - Exercise Sheet 3

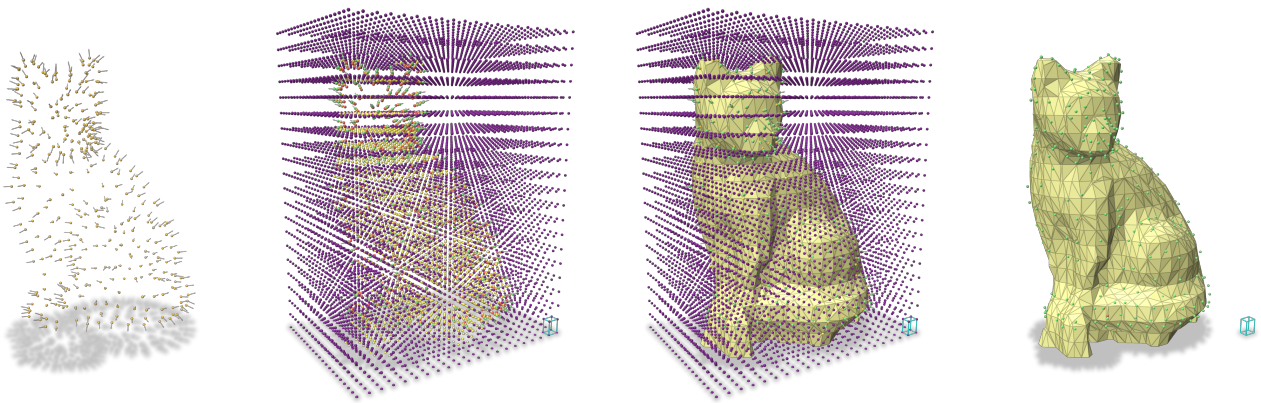
Technische Universität Berlin - Computer Graphics

Handout 04. June 2026, Practice Due 24. June 2026, Theory Due 24. June 2026 11:59 pm

Hendrik Meyer, Prof. Dr. Marc Alexa

Exercise 1: Approximation and tessellation of implicit surfaces (7 + 7 Points)

In this exercise you will reconstruct an implicit surface from a 3D point cloud (with given normals). The implicit representation is then converted to a triangle mesh using the marching cubes algorithm. The following is an illustration of the exercise:



The implementation steps are:

1. Read 3D points and their normals from an *.off file. Visualize the points as points and the normals as line segments or arrows. (1 Point)
2. Add the constraints (points $\mathbf{p}_i$ with function values $f(\mathbf{p}_i)$) for the approximation of the implicit function $f(x, y, z)$ using the n input points and normals. For each of input point, you need to add two additional points by displacing it in both the positive and the negative normal direction. This results in a total of $3n$ points. The constraints should be obtained as follows: (1 Point)
 - Compute a (slightly bigger) bounding box of the point cloud.
 - Assign an appropriate value to α (for example $\alpha = 0.01 \times$ bounding box diagonal), which is the distance to the offset points. What effect will a too big or too small α have?
 - Generate the missing $2n$ points by calculating the offset points in positive and negative normal directions. For each point $\mathbf{p}_i$, calculate the corresponding point $\mathbf{p}_{i+n} = \mathbf{p}_i + \alpha \mathbf{n}_i$ ($\mathbf{n}_i$ = the unit normal of point $\mathbf{p}_i$), and check (in logarithmic time) whether $\mathbf{p}_i$ is the closest input point to $\mathbf{p}_{i+n}$. If not, divide α by 2 and recalculate $\mathbf{p}_{i+n}$ until this is the case. Repeat the same procedure with $-\alpha$ so $\mathbf{p}_{i-n} = \mathbf{p}_i - \alpha \mathbf{n}_i$ and the closest input point to $\mathbf{p}_{i-n}$ is $\mathbf{p}_i$.
 - Assign function values to each point. For the original n points the value $f(\mathbf{p}_i) = 0$, as they should be on the surface and $f(\mathbf{p}_{i+n}) = \alpha$ and $f(\mathbf{p}_{i-n}) = -\alpha$ for the added points (α may vary for some points if you needed to change α for them).
3. Divide the bounding box into a changeable grid of $N_X \times N_Y \times N_Z$ uniform cells. Compute the implicit function values for all grid points with the help of all $3n$ constraints using a weighted least squares approximation. In this exercise it is enough to use a constant polynomial basis. The radius of the Wendland function should be changeable again. Display the grid points and make the inside points distinguishable from the outside points (e.g., by coloring according to the sign of the implicit function values). (2 Point)
Additionally you can implement a linear or quadratic polynomial basis. (1 Bonus point)

-
4. Triangulate the implicit function with the marching cubes algorithm (you can find the look-up table for example on Paul Bourke's webpage: <http://paulbourke.net/geometry/polygonise/>). (2 Points)
Implement the marching cubes algorithm on the GPU, for example using CUDA, OpenCL, or OpenGL Compute Shaders. (1 Bonus point)
 5. Calculate an approximate normal for each vertex of the triangulation from the underlying implicit representation using finite differences. (1 Points)

There are two bonus exercises:

1. Implement the extended marching cubes algorithm.¹ (1 Bonus point)
2. Render an image of the implicit surface using ray casting and visualize the intersection points in 3D. The rays can be generated for example using `glm::unProject(...)` or in the case of polyscope using `polyscope::view::screenCoordsToWorldRay(...)`. Implement the ray marching as well as a sphere tracing version. (1 Bonus point for each)

You will receive one additional point for each bonus exercise that you implement on the GPU.

Exercise 2: Theory (4 + 1 Points)

1. What conditions should a function $f(\mathbf{x})$ satisfy in order to regularly describe an implicit surface? (1 Point)
2. Given two circles as conic section with arbitrary midpoints and radii. What is the (algebraic) sum of the two functions whose zero level sets are the conic sections? (1 Point)
3. Given an implicit function as modeled and described in the practice part (using approximation of constant polynomials), describe the range of this function. (1 Point)
4. Does the marching cubes algorithm lead to aliasing artifacts? Please explain your answer. (1 Point)
5. Compute the gradient of the implicit function, which is approximated by constant polynomials. (1 Bonus point)

¹Kobbelt, L. P., Botsch, M., Schwannecke, U., & Seidel, H.-P. (2001). Feature sensitive surface extraction from volume data. In Proceedings of the 28th annual conference on Computer graphics and interactive techniques - SIGGRAPH '01 (pp. 57–66). New York, New York, USA: ACM Press. doi:10.1145/383259.383265